

Modélisation des données SWH

Jeudi 28 mars 2024

Le modèle de données Cassandra

Cassandra organise les données en **tables** dénormalisées

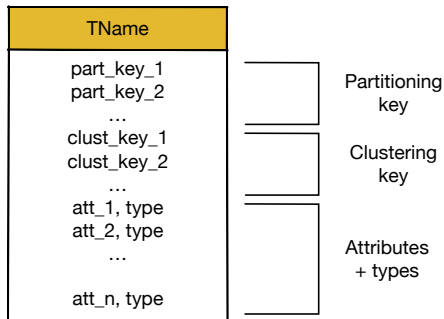
Principales différences avec le modèle relationnel

- Pas de clé étrangère (et pas de jointure)
- Imbrication (*nesting*) de valeurs structurées
- Pas d'indépendance logique/physique : la clé détermine le placement
- Sauf exception, les recherches se font sur la clé

Principe de conception

Pas de jointure, donc pas de symétrie d'accès : on doit organiser les données en fonction des accès applicatifs.

Structure d'un schéma



```
create table Tname (  
    part_key_1 type,  
    part_key_2 type,  
    ... type,  
    clust_key_1 type,  
    ... type,  
    att_1 type,  
    att_2 type,  
    ... type,  
    primary key (  
        (part_key_1, part_key2, ...),  
        (clust_key_1, clust_key_2, ...)  
    )  
)
```

Imbrication de structures

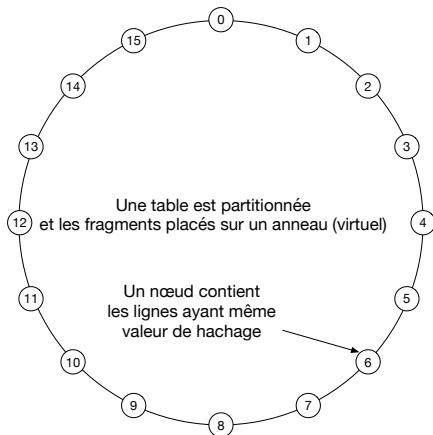
C'est le principe de dénormalisation : on regroupe les données le plus possible dans des lignes pour éviter les jointures.

Une valeur d'attribut peut correspondre :

- à un ensemble de valeurs (non ordonnées) : **SET**
- à une liste de valeurs : **LIST**
- à un dictionnaire : **MAP**
- à un nuplet : **TUPLE**
- à une instance d'un type utilisateur

Les ensembles doivent rester de taille raisonnable

Clé de partitionnement = placement



- Pour chaque ligne on applique une fonction de hachage aux valeurs de la clé de **partitionnement**
- La valeur de hachage détermine le placement dans le système distribué
- $\Rightarrow$ une requête incluant comme critère une clé de partitionnement ne concerne qu'un seul serveur

Clé de regroupement = contiguité

TName		
Part key	Clust key	(key, val) pairs
www.swh.org	aaa.html	...
www.swh.org	bird.jpg	...
...	...	...
www.swh.org	main.css	...
www.swh.org	...	...
www.swh.org	zeta.zip	...

- Dans un fragment (sur un serveur) les lignes sont triées sur la clé primaire
- Pour une même valeur de partitionnement, les lignes sont **consécutives** et ordonnées sur la clé de regroupement
- $\Rightarrow$ une requête sur une préfixe de la clé primaire (part. + regroupement) correspond à un parcours **séquentiel** sur un seul nœud

Principes de conception

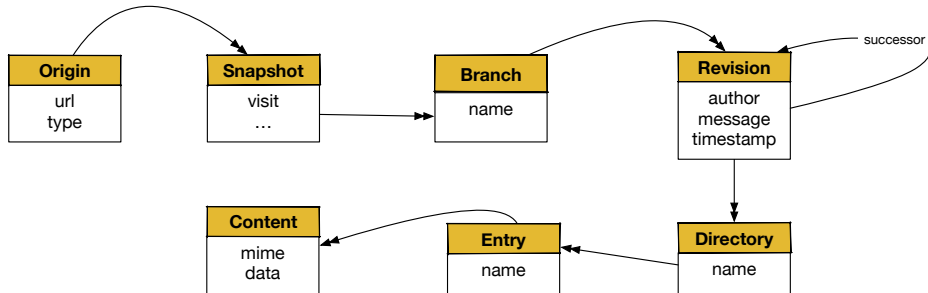
On adopte un des nombreux schémas possibles en fonction des besoins.

- On conçoit les **chemins d'accès**, ou séquence des requêtes émises par une application
- On organise les tables pour que chaque requête s'exécute **localement** et **séquentiellement**
- On peut créer ponctuellement des index ou des vues matérialisées
- Au pire (mais inévitable ?) on multiplie les organisations physiques et donc la redondance

Pour quelle type d'application ?

Acceptable pour une approche WORM (*Write Once, Read many*)

Notre modèle



Pour chaque table on a un point d'accès (pe 'origin') et l'identification relative à ce point d'accès (pe 'snapshot'). On nommera la table `snapshot_by_origin`

Quelques exemples : cherchons les visites

Une première approche purement relationnelle ne fonctionne pas.

Schema

```
create table Origin
(
  origin_id uuid,
  url varchar,
  description text,
  primary key (origin_id))

create table Visit
(
  visit_id uuid,
  origin_id
  visit_ref int,
  date date,
  primary key (visit_id) )
```

Queries

Il faut deux requêtes (pas de jointure)

```
select origin_id in oid
from Origin
where url = 'sw.h.org'
```

```
select * from Visit
where origin_id = oid
```

Et Cassandra refuse les deux !

Organisons les clés et chemins d'accès

Les critères de recherche doivent être dans la clé

```
create table Origin
(url varchar,
 description text,
 primary key (url)
)
```

```
create table Visit_by_origin
(url_origin varchar,
 visit_ref int,
 date date,
 primary key (url_origin,
              visit_ref)
)
```

```
select * from Origin
where url = 'swh.org'
```

```
select * from Visit_by_origin
where url = 'swh.org'
```

```
select * from Visit_by_origin
where url = 'swh.org'
and visit_ref < 3
```

Imbriquons pour avoir moins de tables

Un type Visit

```
create type Visit (  
    number int,  
    date_visit date)
```

Imbriqué dans Origin

```
create table Origin  
    (url varchar,  
     description text,  
     visites list<Visit>,  
     primary key (url)  
    )
```

Suppose un nombre limité de visites.

Une requête localisée et unique pour trouver toutes les visites d'une origine.

```
select * from Origin  
where url = 'swb.org'
```

Les origines visitées au moins 10 fois

```
select * from Origin  
where url = 'swb.org'  
and count(visit.number) >= 10
```

Tentons une modélisation complète : Snapshot

```
create type Origin (  
    description int,  
    other_infos text)  
  
create table Snapshot_by_origin  
    (url varchar,  
    snapshot_date,  
    snapshot_id uuid,  
    origin Origin,  
    visits list<Visit>,  
    branches map<string, uuid>,  
    primary key (url,  
    snapshot_date) )
```

Avec peu de branches par snapshot

Tous les snapshots d'une origine.

```
select * from Snapshot_by_origin  
where url = 'sw.h.org'
```

Snapshots après une date et contenant
une branche master

```
select * from Snapshot_by_origin  
where url = 'sw.h.org'  
and snapshot_date > '01-MARCH-2024'  
and branches contains key 'master'
```

On obtient les id des snapshots et des
branches

Connaissant une branche : les Revisions

Ici, vraie structure de graphe. Pas idéal...

```
create table Revision_by_branch
(branch_id uuid,
 revision_id uuid,
 parents list<uuid>,
 author varchar,
 message varchar,
 tstamp date,
 primary key (branch_id,
 revision_id)
)
```

Toutes les révisions d'une branche par Stefano.

```
select * from Revision_by_branch
where branch_id = 'bxyz'
and author = 'Stefano'
```

Les révisions dont un parent est la révision 'abcd'

```
select * from Revision_by_branch
where branch_id = 'bxyz'
and parents contains 'abcd'
```

On obtient les id des révisions d'une branche

Connaissant une Revision : les Directory

Idem, structure d'arbre (?), une requête par directory. Pas idéal...

```
create table Directory_by_revision
(revision_id uuid,
 directory_id uuid,
 children list<uuid>,
 parent_id uuid,
 primary key (revision_id,
              directory_id)
)
```

On peut naviguer parent/enfants, mais une requête à chaque fois

À l'inverse si je veux avoir la liste des révisions d'une directory, je dois inverser la clé...

```
create table Revision_by_directory
(directory_id uuid,
 revision_id uuid,
 children list<uuid>,
 primary key (revision_id,
              directory_id)
)
```

Revision_by_directory peut être une **vue matérialisée**

Connaissant une Directory : les Entry

```
create table Entry_by_dir
(directory_id uuid,
 name varchar,
 access_rights varchar,
 content_ref varchar,
 primary key (directory_id,
 name)
)
```

NB : sans doute trop d'entrées par directory pour les représenter par une liste imbriquée

Toutes les entrées d'une directory

```
select name from Entry_by_dir
where directory_id = 'xyz'
```

Une entrée README.md

```
select content_ref from Entry_by_dir
where directory_id = 'xyz'
and name = 'README.md'
```

En résumé (provisoire)

En synthèse :

- La clé primaire détermine le stockage *et* les requêtes possibles
⇒ **clé de partitionnement** = point d'accès, **clé de regroupement** = identifiant relatif au point d'accès
- Il faut parfois “retourner” une clé primaire si on souhaite un accès symétrique à une table existante (cf. exemple révision/Directory)
⇒ automatisable avec une vue matérialisée (forte redondance)
- L'imbrication d'ensembles ou listes peut limiter le nombre d'étapes
⇒ leur taille doit être restreinte, cf. exemple des branches